

SALESTORM

High-Scale E-Commerce Sale

➤ REQUIREMENTS & ASSUMPTIONS

1. Problem Statement

Design a scalable e-commerce flash-sale system that can handle **10,000 concurrent purchase requests for only 100 available units**, while preventing overselling, duplicate reservations, and duplicate payments.

2. Functional Requirements

- Product discovery and browsing
- Cart management
- Inventory availability checking
- Temporary inventory reservation
- Reservation expiry and release
- Checkout and payment processing
- Order creation and lifecycle management
- Shipment and delivery tracking
- Customer notifications
- Idempotent handling of duplicate requests

3. Non-Functional Requirements

- Handle high concurrent traffic and sudden flash-sale spikes
- Prevent inventory overselling
- Maintain strong consistency for inventory operations
- Provide high availability and fault tolerance
- Support horizontal scaling
- Ensure secure payment processing
- Provide logging, monitoring, and distributed tracing

4. Strict Guarantees

- Inventory must never become negative
- Maximum successful sales must not exceed available stock

- Duplicate requests must not create duplicate reservations, payments, or orders
- Expired/failed reservations must release inventory
- Order and reservation state transitions must remain valid

5. Assumptions

- Flash-sale stock: **100 units**
- Concurrent purchase requests: **10,000**
- Maximum quantity per successful purchase: **1 unit**
- Reservation TTL: **5 minutes**
- Payment is handled through an external payment gateway
- Core transactional data is stored in a relational database
- Downstream operations use asynchronous messaging where appropriate

6. Performance Targets

- Support approximately **10,000 requests/second** under normal operation
- Design for flash-sale spikes up to approximately **500,000 requests/second**
- Scale horizontally during traffic surges
- Maintain low response latency for critical purchase operations

7. Critical Dependencies

- Relational Database
- Payment Gateway
- Message Queue
- Redis/Cache
- Monitoring and Logging System

8. Success Criteria

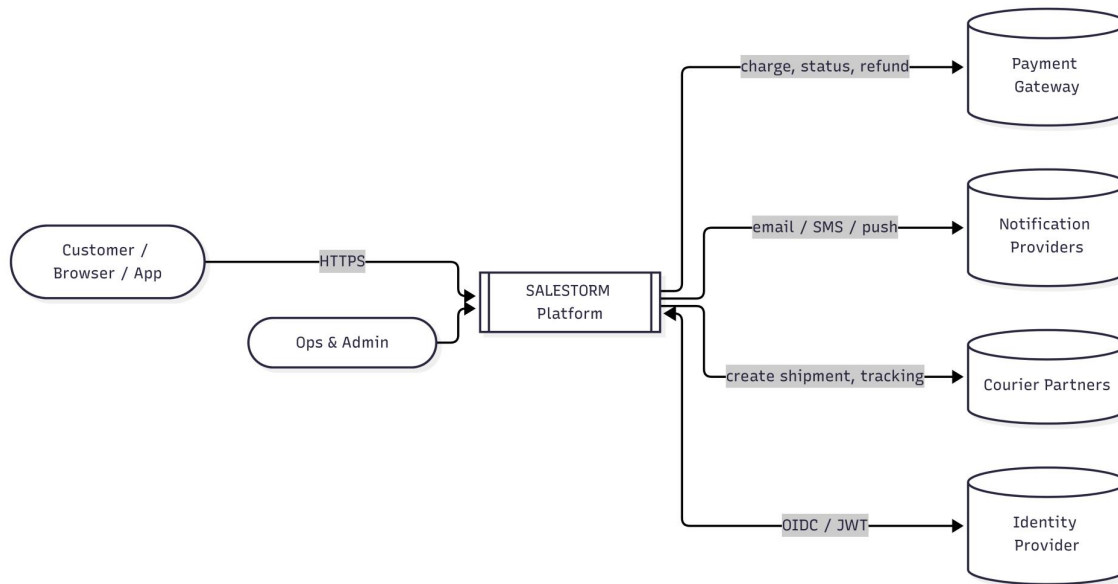
The system must successfully control high traffic, sell **at most 100 units**, prevent overselling and duplicate transactions, recover from failures, and maintain a consistent order lifecycle.

➤ HIGH-LEVEL ARCHITECTURE (HLD)

The SALESTORM system follows this high-level flow:

Customer → CDN/WAF → Load Balancer → API Gateway → Backend Services → Database / External Systems

System Context Diagram

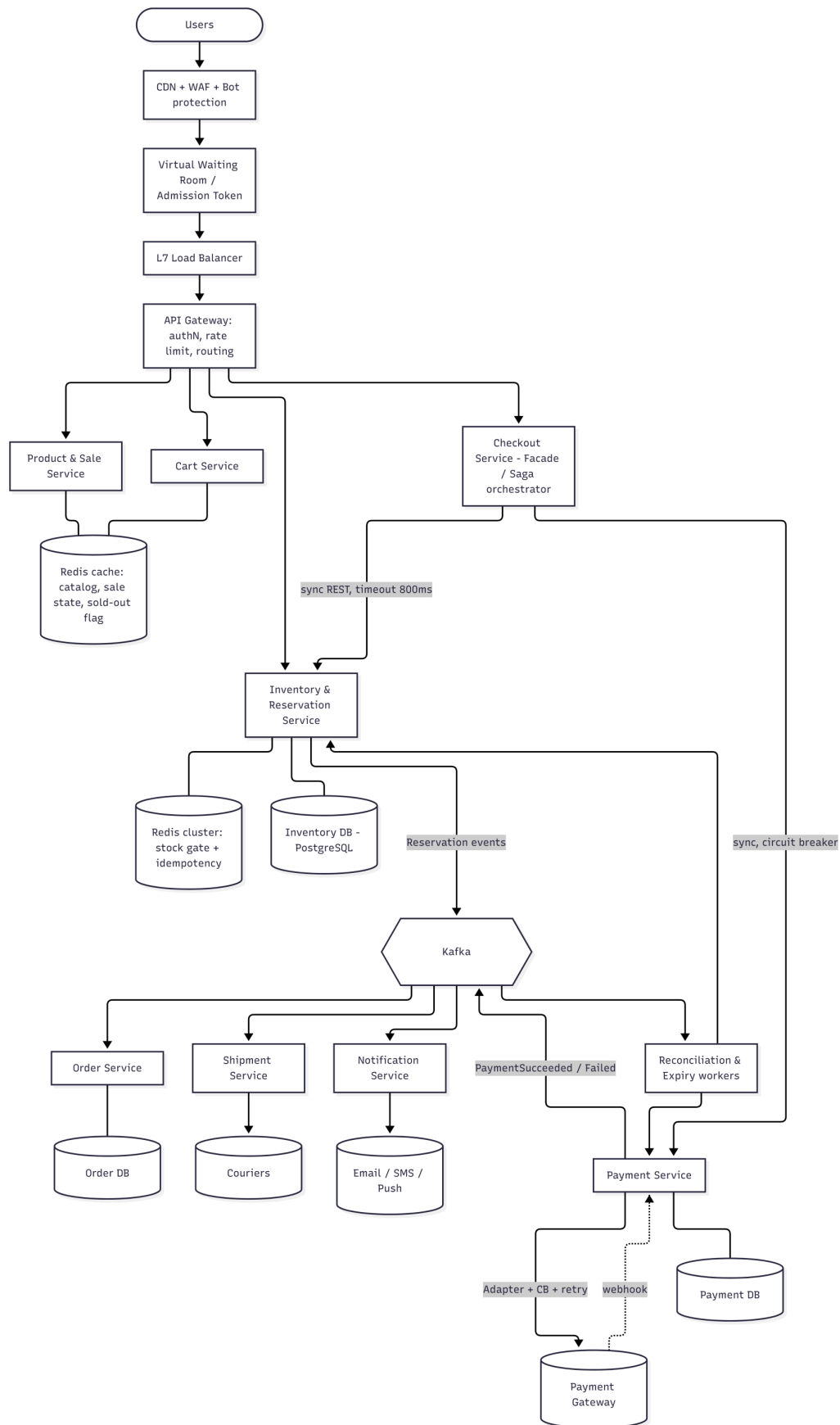


Container / Service Architecture

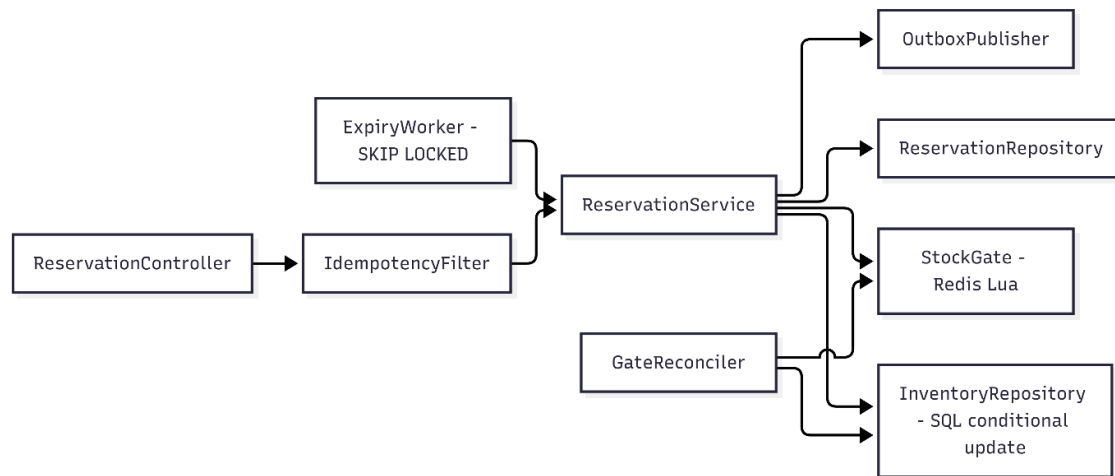
The SALESTORM platform is designed as a modular service-based architecture, where each service has a clearly defined responsibility.

Main Components

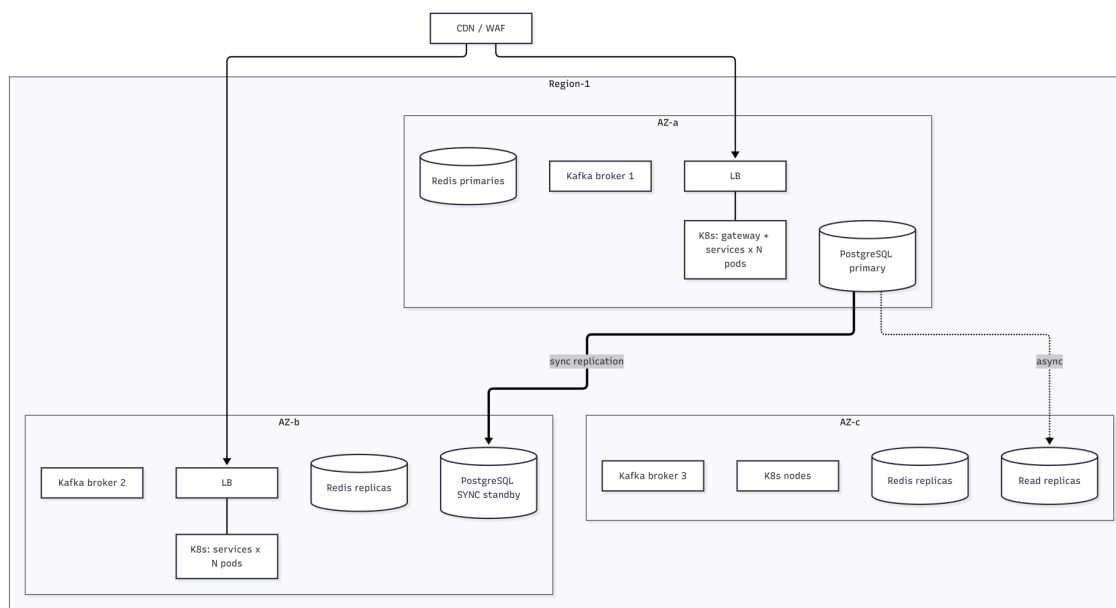
- **CDN + WAF:** Handles incoming traffic, caches static content, and provides protection against malicious or excessive requests.
- **Virtual Waiting Room:** Controls the number of users entering the purchase flow during extreme traffic spikes.
- **Load Balancer:** Distributes requests across multiple service instances for horizontal scalability.
- **API Gateway:** Acts as the single entry point and handles authentication, rate limiting, request validation, and routing.
- **Product & Sale Service:** Provides product details, sale information
- **Cart Service:** Manages customer carts and cart items.
- **Inventory & Reservation Service:** Manages stock availability and temporary reservations. It is the main consistency boundary for inventory.
- **Checkout Service:** Coordinates the checkout process and communicates with Inventory and Payment services.
- **Payment Service:** Processes payments through an external payment gateway and handles payment success, failure, timeout, and duplicate requests.
- **Order Service:** Creates and manages orders after successful payment.



Component Diagram - Inventory & Reservation Service



Deployment Diagram



Autoscaling: HPA based on CPU, request rate and Kafka consumer lag.

Pre-warming: Scale services before the flash sale begins.

Stateless Services: Application state is maintained in Redis, PostgreSQL and Kafka.

Component	Why	Bottleneck & Mitigation
CDN/WAF	Absorb traffic and block malicious requests	Cache tuning and bot protection
Waiting Room	Converts traffic spike into controlled flow	Token-based admission

Component	Why	Bottleneck & Mitigation
API Gateway	Authentication, rate limiting and routing	Stateless horizontal scaling
Redis Stock Gate	Rejects requests before DB contention	Atomic operations and stock partitioning if needed
Inventory DB	Durable inventory source of truth	Conditional updates and product-based partitioning
Kafka	Decouples order, shipment and notification	Partitioning and DLQ
Payment Adapter	Isolates external payment gateway	Timeout, circuit breaker and reconciliation

➤ LOW-LEVEL DESIGN (LLD)

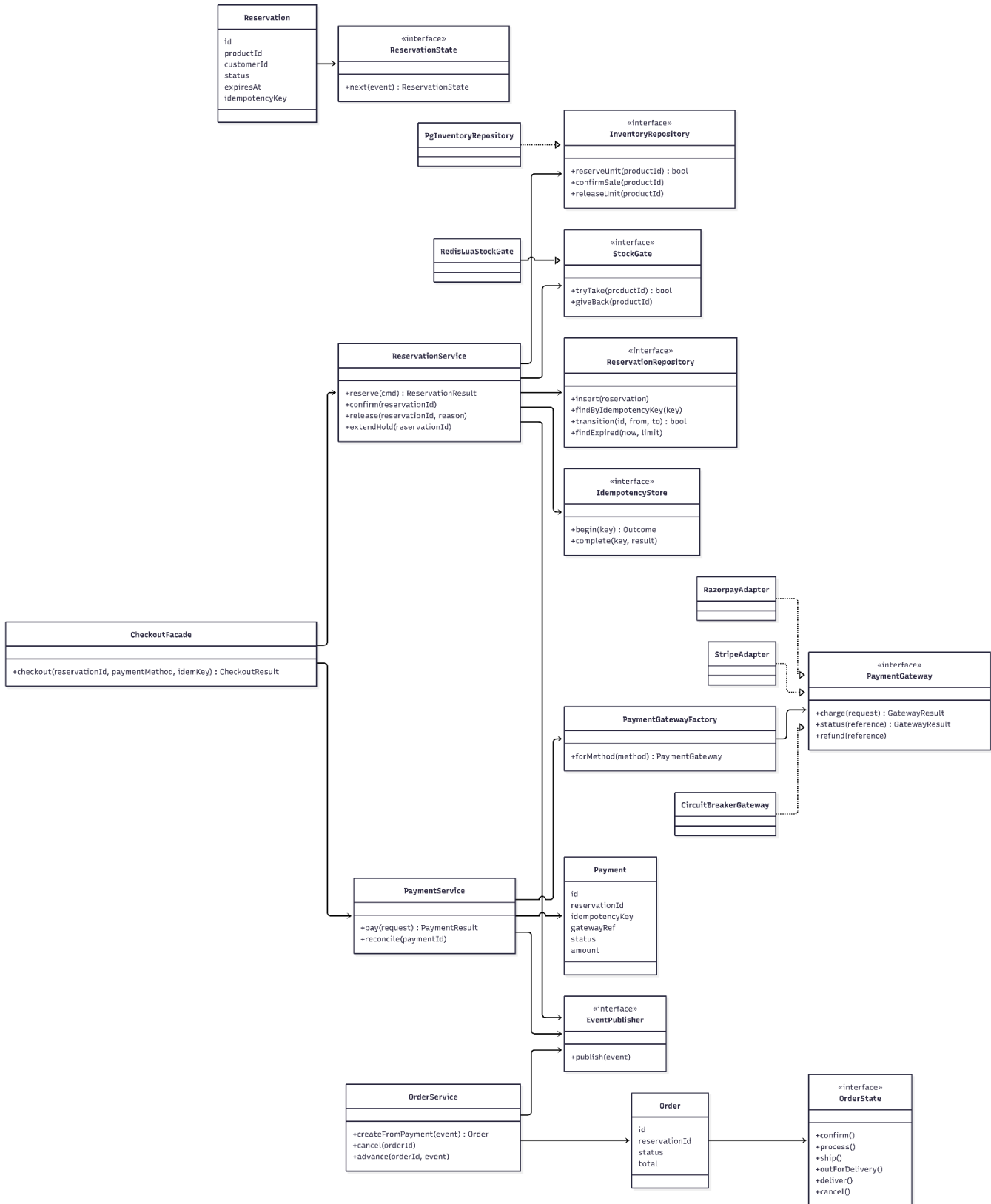
The LLD defines the internal structure of the critical SALESTORM modules: Inventory/Reservation, Payment, and Order, along with the Checkout Facade.

The design separates responsibilities using services, repositories, interfaces, adapters, and state managers. This keeps the system modular, testable, and easier to maintain.

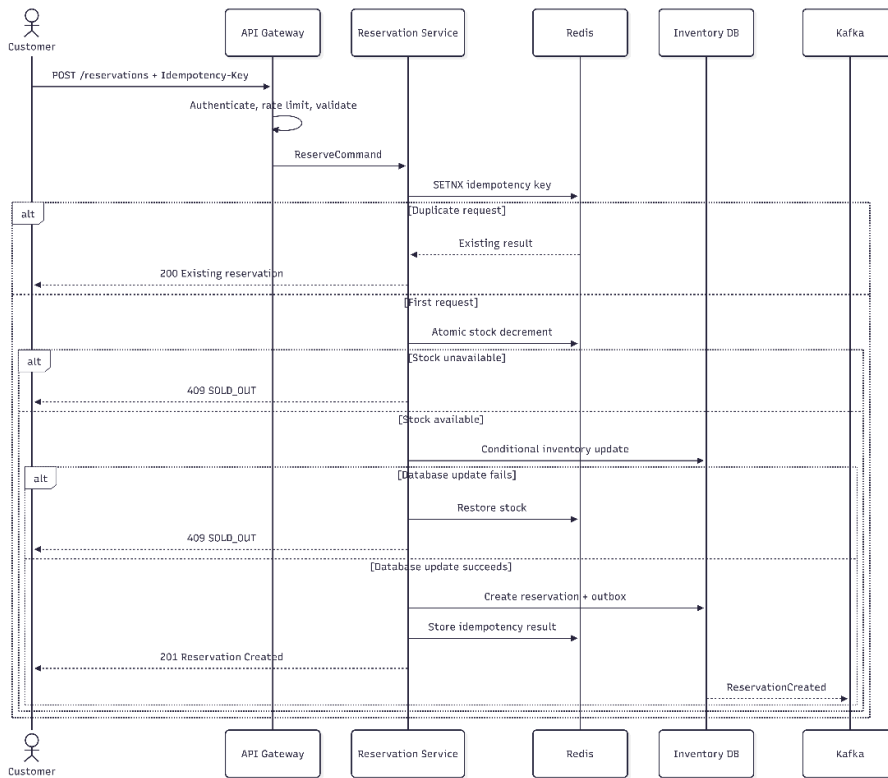
Key Design Principles

- Inventory/Reservation: Handles stock reservation, release, expiry, and idempotency.
- Payment: Handles payment processing, gateway integration, failures, timeouts, and reconciliation.
- Order: Handles order creation and controlled state transitions.
- Checkout Facade: Coordinates reservation and payment operations.
- Repository Pattern: Separates business logic from database operations.
- Adapter Pattern: Provides a common interface for different payment gateways.

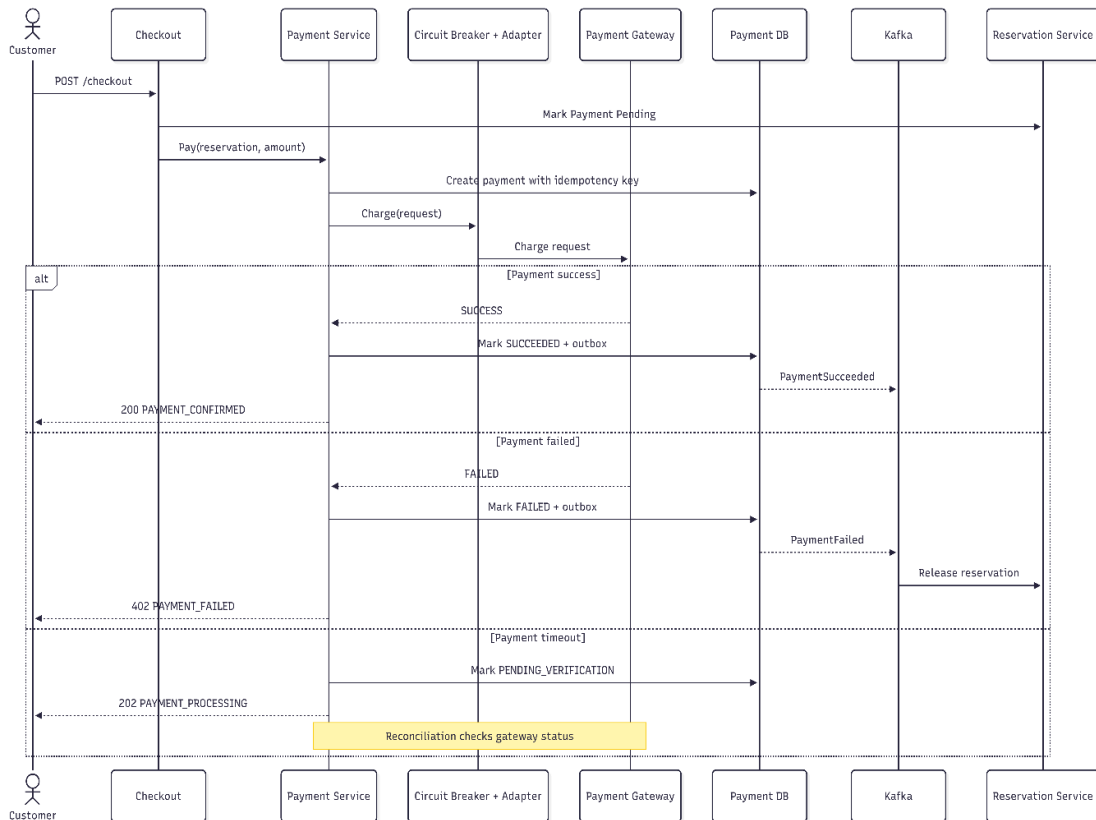
Class Diagram



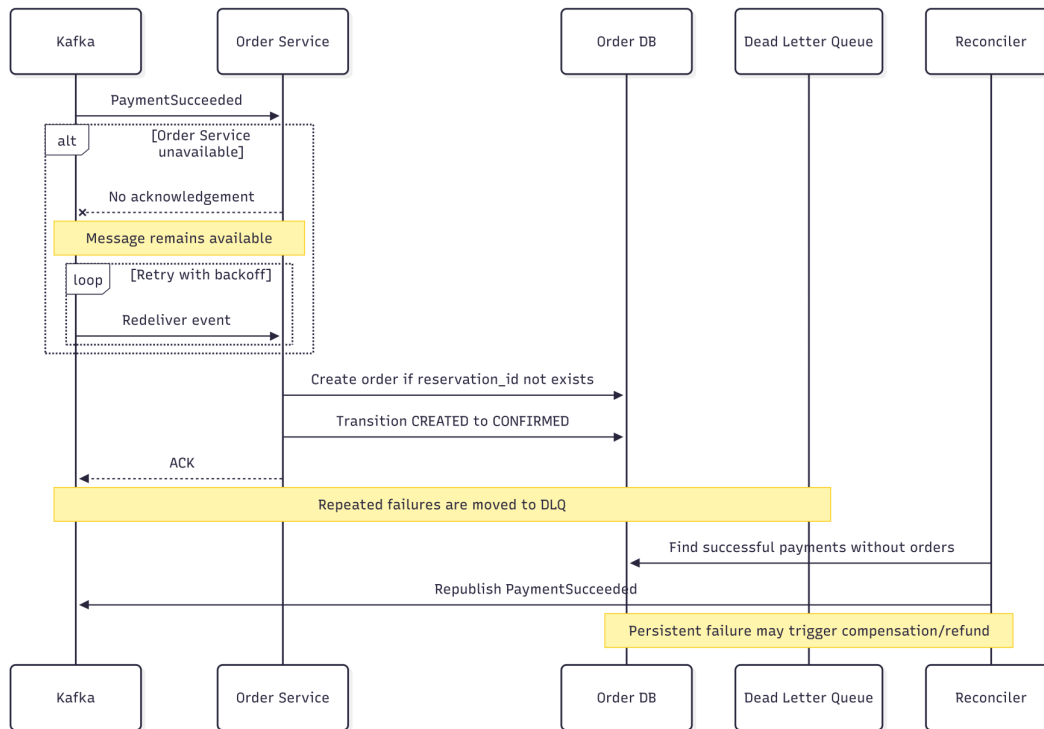
Reservation Sequence



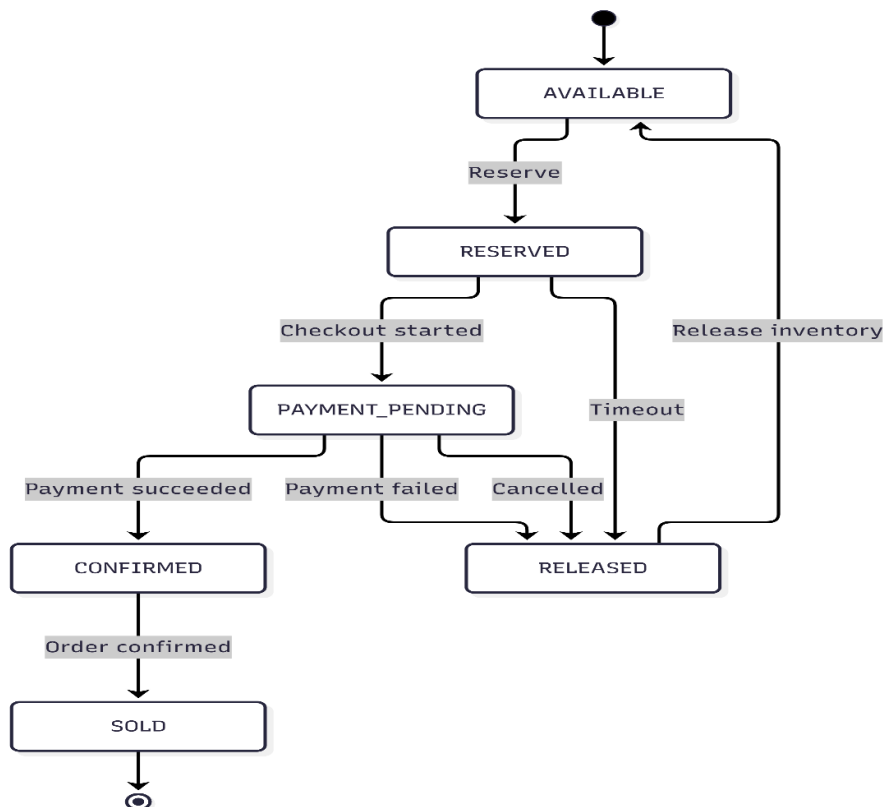
Payment Sequence



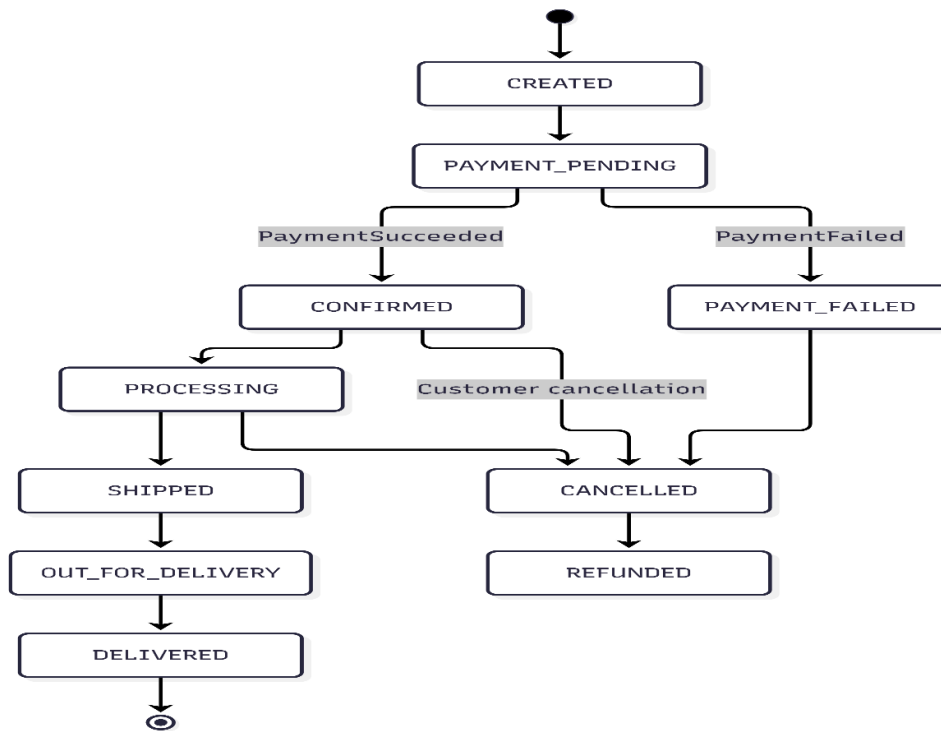
Order Creation & Recovery



Reservation State Diagram

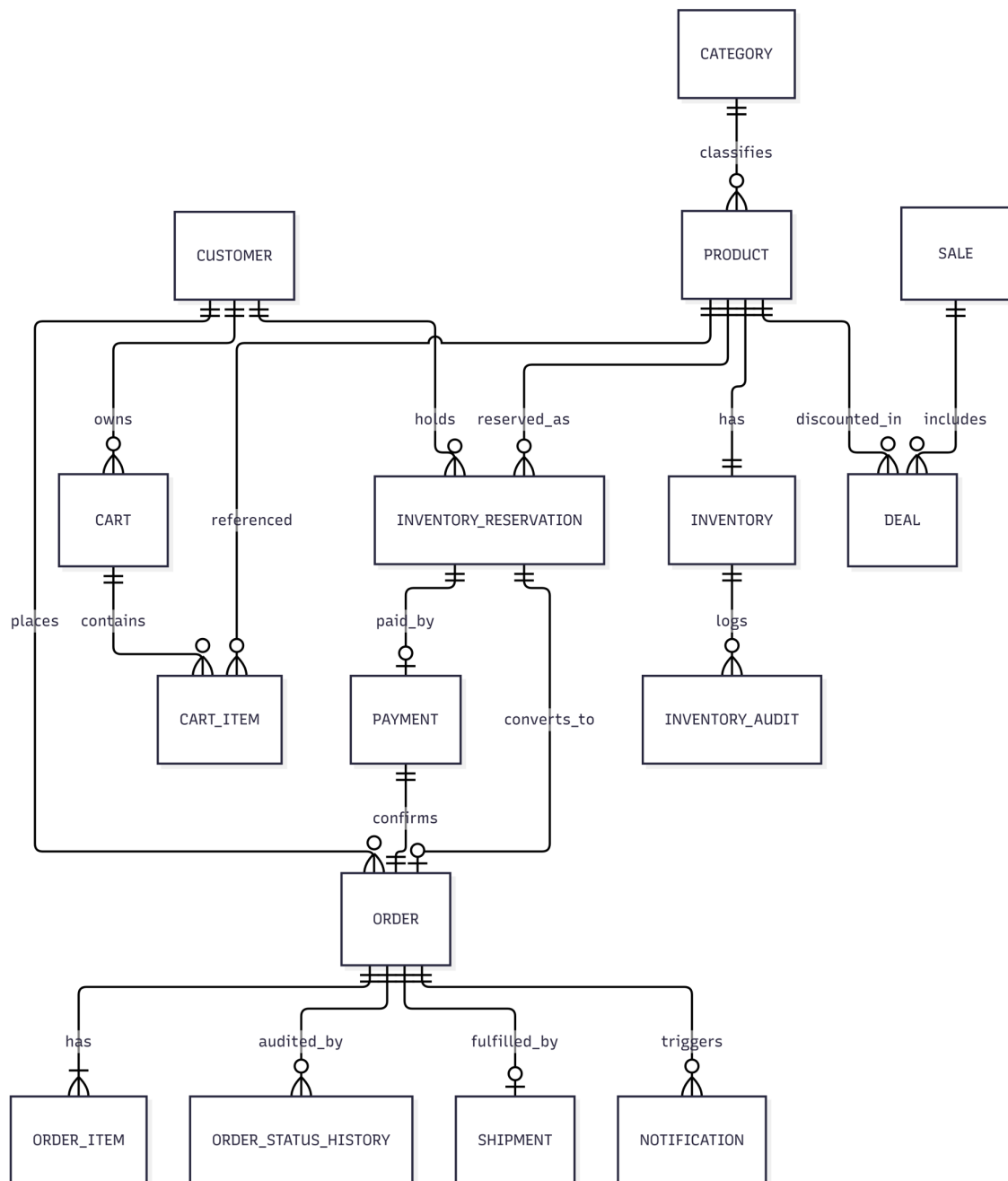


Order State Diagram



Concern	Location
Input validation	Controller / Command Validator
Idempotency	API Idempotency Filter + DB unique constraints
Error handling	Global Exception Mapper
Business rules	Domain/Service Layer
Inventory consistency	Inventory Service + Database
Payment reliability	Payment Service + Circuit Breaker + Reconciliation
Order state validation	Order State Manager + DB conditional update

ER DIAGRAM



➤ SOLID PRINCIPLES & DESIGN PATTERNS

SOLID Principles

Principle	SALESTORM Application
S – Single Responsibility	ReservationService handles reservations, PaymentService handles payments, and OrderService handles orders.
O – Open/Closed	New payment gateways can be added through PaymentGateway without modifying PaymentService.
L – Liskov Substitution	RazorpayAdapter and StripeAdapter can be used wherever PaymentGateway is expected.
I – Interface Segregation	Small interfaces such as PaymentGateway, InventoryRepository, and StockGate avoid unnecessary dependencies.
D – Dependency Inversion	Services depend on interfaces such as repositories and payment gateways instead of concrete implementations.

Design Patterns

Pattern	Usage in SALESTORM
Repository	Separates business logic from PostgreSQL data access.
Adapter	Provides a common interface for different payment gateways.
Factory	Selects the required payment gateway implementation.
State	Manages Reservation and Order lifecycle states.
Facade	CheckoutFacade coordinates reservation, payment, and order operations.
Strategy	Allows different business strategies such as payment or pricing policies.
Circuit Breaker	Prevents repeated calls to an unhealthy payment gateway.
Observer / Event-driven	Kafka events notify Order, Shipment, and Notification services asynchronously.

Key Benefit

SOLID principles keep the system **modular, maintainable, testable, and extensible**, while design patterns provide reusable solutions for payment integration, state management, data access, and fault handling.

➤ API & EVENT SPECIFICATION

Key REST APIs

API	Method	Purpose
/products/{id}	GET	Get product details
/cart/items	POST	Add product to cart
/reservations	POST	Reserve inventory
/reservations/{id}/release	POST	Release reservation
/checkout	POST	Start checkout
/payments	POST	Initiate payment
/payments/{id}	GET	Check payment status
/orders/{id}	GET	Get order details

Important API Rules

- Use **JWT authentication** for protected APIs.
- Use an **Idempotency-Key** for reservation and payment requests.
- Validate request data before processing.
- Return consistent HTTP status codes and error responses.

Important Kafka Events

Event	Producer	Consumer
ReservationCreated	Inventory Service	Checkout
ReservationExpired	Inventory Service	Order/Inventory
PaymentSucceeded	Payment Service	Order Service
PaymentFailed	Payment Service	Inventory Service
OrderCreated	Order Service	Shipment/Notification
ShipmentUpdated	Shipment Service	Order/Notification

Event Reliability

Events are published using a **transactional outbox**. Consumers use **idempotent processing**, retries, and a **DLQ** for failed messages.

➤ SCALABILITY & RELIABILITY

Scalability

- Use **horizontal scaling** with stateless service instances.
- CDN + WAF handles high incoming traffic.
- **Virtual Waiting Room** controls flash-sale traffic.
- Redis provides fast caching and inventory gating.
- Kafka absorbs asynchronous workloads.
- Database uses read replicas and connection pooling.
- Auto-scaling based on CPU, request rate, and Kafka consumer lag.

Reliability

- **Timeouts** prevent requests from hanging.
- **Retries** handle temporary failures.
- **Circuit Breaker** protects unhealthy external services.
- **Idempotency** prevents duplicate payments/orders.
- **DLQ** stores repeatedly failed events.
- **Reconciliation workers** recover inconsistent payment/order states.
- **Reservation expiry** releases unpaid inventory.

Flash-Sale Target

Design for **10,000 concurrent requests**, while ensuring that only the available **100 units** can be successfully purchased.

➤ CONCURRENCY CONTROL & INVENTORY RESERVATION

Problem

Handle **10,000 concurrent requests for 100 units** without overselling or negative inventory.

Inventory Lifecycle

AVAILABLE → RESERVED → PAYMENT_PENDING → CONFIRMED → SOLD

Failure: **RESERVED/PAYMENT_PENDING → RELEASED**

Reservation Flow

1. Request arrives with an **Idempotency Key**.
2. Inventory Service validates the request.

3. **Redis Stock Gate** performs an atomic stock check.
4. If stock exists, the database performs a **conditional inventory update**.
5. A reservation is created with a **5-minute expiry**.
6. Successful payment confirms the reservation.
7. Failed/expired reservations release the inventory.

Concurrency Control

Database Row Locking: Strong consistency but can cause high contention.

Conditional Update: Avoids long locks and performs better under concurrency.

Selected Approach: Use **Redis atomic stock gating + PostgreSQL conditional update**. Redis provides fast rejection, while PostgreSQL remains the source of truth.

Idempotency

A unique idempotency key ensures repeated requests return the existing result instead of creating duplicate reservations.

Reservation Expiry

Unpaid reservations automatically expire after the TTL and release inventory back to the available pool.

Key Guarantee

For **100 units and 10,000 requests**, a maximum of **100 reservations can succeed**. All other requests receive **SOLD_OUT**, ensuring no overselling or negative inventory.

➤ SECURITY & OBSERVABILITY

Security

- **Authentication:** OIDC/JWT-based authentication.
- **Authorization:** Role-based access control (RBAC).
- **API Security:** HTTPS, WAF, rate limiting, input validation, bot protection.
- **Payment Security:** Do not store card details; use payment-gateway tokens.
- **Data Security:** Encrypt data in transit and at rest.
- **Secrets:** Store credentials/API keys in a secrets manager.
- **Idempotency:** Prevent duplicate reservations, payments, and orders.
- **Audit:** Log security-sensitive actions and state changes.

Observability

- **Metrics:** RPS, latency, error rate, inventory remaining, reservation success/failure, payment success rate, Kafka lag.
- **Logging:** Centralized structured logs with correlation/request IDs.
- **Tracing:** Distributed tracing across Gateway → Inventory → Payment → Order.
- **Alerts:** Trigger on high error rate, latency, payment failures, DB issues, Redis failures, and Kafka lag.
- **Dashboards:** Real-time sale health, inventory status, traffic, payments, and service health.

Key Goal

Maintain **secure transactions, traceable failures, and real-time visibility** during the 10K-user flash-sale spike.

➤ ADR & TRADE-OFFS

Key Architecture Decisions

Decision	Choice	Reason
Inventory consistency	PostgreSQL + conditional update	Prevent overselling
Fast stock control	Redis atomic operations	Fast rejection under heavy load
Traffic spike	Waiting Room + Rate Limiting	Protect backend services
Async processing	Kafka	Decouple services and improve resilience
Payment integration	Adapter + Circuit Breaker	Isolate gateway failures
Duplicate requests	Idempotency keys	Prevent duplicate operations
Event reliability	Transactional Outbox	Avoid DB/event inconsistency
Reservation expiry	Background worker	Automatically release abandoned stock

Major Trade-offs

- **Strong consistency vs performance:** PostgreSQL is the source of truth; Redis provides fast admission control.
- **Synchronous vs asynchronous:** Critical inventory/payment decisions are synchronous; downstream processing is asynchronous.

- **Availability vs consistency:** During inventory uncertainty, reject requests rather than risk overselling.
- **Complexity vs scalability:** Kafka, Redis, reconciliation, and idempotency add complexity but support flash-sale scale.
- **Cost vs reliability:** Multi-AZ deployment and replicas increase infrastructure cost but reduce downtime.

Key Principle

Correctness is prioritized over throughput: when the system cannot guarantee inventory consistency, it should fail safely rather than oversell.

➤ 10K USERS / 100 STOCK STRESS SCENARIO

Scenario

- **Concurrent users:** 10,000
- **Available stock:** 100 units
- **Purchase limit:** 1 unit/customer
- **Reservation TTL:** 5 minutes

Request Flow

1. Users enter through **CDN/WAF + Waiting Room**.
2. API Gateway authenticates and rate-limits requests.
3. Inventory Service receives reservation requests.
4. **Redis atomic stock gate** immediately rejects requests when stock is unavailable.
5. PostgreSQL performs the **conditional inventory update**.
6. Only successful requests receive reservations.
7. Payment confirms valid reservations.
8. Failed/expired reservations release stock.

Expected Result

Metric	Expected Result
Concurrent users	10,000
Available units	100
Maximum successful reservations	100

Metric	Expected Result
Overselling	0
Negative inventory	0
Duplicate payments	0
Expired holds	Automatically released

Failure Handling

- **Redis failure:** Fail closed or temporarily reject reservations.
- **Database failure:** Do not confirm reservation; retry safely.
- **Payment timeout:** Mark as PENDING_VERIFICATION and reconcile.
- **Kafka failure:** Events remain available for retry.
- **Service failure:** Stateless services restart/scale horizontally.
- **Expired reservation:** Background worker releases stock.

Jury Walkthrough

10,000 users → controlled admission → atomic stock gate → conditional DB update → 100 reservations maximum → payment → order creation

Final guarantee: No overselling, even under extreme concurrency.

AI USAGE & VALIDATION NOTE

AI Usage

AI was used to assist with:

- Architecture and service decomposition
- UML and Mermaid diagrams
- API and event design
- Concurrency and inventory strategies
- Scalability, reliability, security, and observability considerations
- ADRs and trade-off analysis

Validation

The proposed design was reviewed against the hackathon requirements, especially:

- **10,000 concurrent users**
- **100 units of inventory**

- **Zero overselling**
- **Idempotent reservations and payments**
- **Failure recovery and reconciliation**
- **Horizontal scalability**
- **Secure and observable operations**

Final Principle

AI-assisted design was treated as a starting point. Architecture decisions, concurrency guarantees, failure scenarios, and trade-offs were manually reviewed and validated against the stated requirements.